Object Oriented Programming (CSEG1044)
Unit 04 – Lecture 01 Notes
Generics Fundamentals (Generic Classes)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms **3**

7 Detailed Notes **4**

7.1 Why this topic exists . . . 4
7.2 Attempt 1: separate classes for every type . . . 4
7.3 Attempt 2: use one class with `Object` . . . 5
7.4 Why `Object` is unsafe . . . 5
7.5 Attempt 3: use a generic class . . . 5
7.6 How to read the syntax correctly . . . 6
7.7 Safe usage . . . 6
7.8 Why generic classes improve teaching clarity . . . 6
7.9 Common naming conventions . . . 7
7.10 Primitive types do not work directly . . . 7
7.11 Raw types and why they are dangerous . . . 7
7.12 A useful intuition about compile-time vs runtime . . . 8

8 Running Project Connection **8**

9 Common Mistakes and Confusions **8**

10 Worked Example: campus resources **8**

11 Demo / Observation Task **10**

12 Practice Check (with brief solutions) **10**

13 Self-Study Practice (no solutions) **10**

14 Further Study **11**

15 Exit Questions (with sample answers) **11**

1 Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Main new ideas:** Why this topic exists, Attempt 1: separate classes for every type, Attempt 2: use one class with `Object`
- **Course thread:** Use Generics Fundamentals (Generic Classes) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

2 Lecture Overview

Students usually find generics difficult for the wrong reason. The syntax looks unusual, so they try to memorize angle brackets instead of understanding the design problem.

This lecture starts from the **problem** first:

1. repeated classes for similar logic,
2. a tempting but unsafe `Object`-based shortcut,
3. a generic class that gives both reuse and type safety.

Once that progression is clear, the syntax becomes much easier to read.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Classes and methods**: A generic class is still a class; generics only add type parameters to familiar structures. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Type relationships**: Generic type safety is easier to read once basic inheritance intuition is already clear. Main reference: `Unit 02 / Lecture 04 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Explain why generic classes are better than duplicated classes or `Object`-only designs.
- Read and write a generic class such as `ResourceHolder<T>`.
- Distinguish **type parameter**, **type argument**, and **parameterized type**.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Why this topic exists → Attempt 1: separate classes for every type → Attempt 2: use one class with `Object` → Why `Object` is unsafe → Attempt 3: use a generic class.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

6 Key Terms

- **Generic class**: a class parameterized by one or more types
- **Type parameter**: the placeholder in the class declaration, such as `T`
- **Type argument**: the real type supplied later, such as `Book`
- **Parameterized type**: the typed form of a generic class, such as `ResourceHolder<Book>`
- **Raw type**: the generic class name used without type arguments, for example `ResourceHolder`
- **Wrapper class**: the object form of a primitive, for example `Integer` for `int`

7 Detailed Notes

7.1 Why this topic exists

Imagine that our project stores different campus resources:

- books,
- laptops,
- lab keys,
- maybe later even ID cards or equipment requests.

Now suppose we want a simple class whose job is only this:

> store one item and return one item

The behavior is always the same. Only the data type changes.

7.2 Attempt 1: separate classes for every type

A beginner may write:

- `BookHolder`
- `LaptopHolder`
- `LabKeyHolder`

Each class has the same basic methods:

- store an item,
- return the stored item.

What is good here?

- The code is type-safe.
- A `BookHolder` clearly stores books only.

What is the problem?

- The class logic is repeated.
- Every new data type requires another almost identical class.
- Maintenance becomes wasteful.

So separate classes give safety, but they do not scale well.

7.3 Attempt 2: use one class with `Object`

To remove duplication, many students try this design:

```
class ResourceHolder {
    private Object item;

    void set(Object item) { this.item = item; }
    Object get() { return item; }
}
```

Why this seems attractive:

- there is now only one class,
- `Object` can refer to many kinds of values,
- code duplication is reduced.

But what do we lose?

- the class no longer knows what it is really storing,
- callers must cast the result,
- wrong casts are detected late at runtime.

7.4 Why `Object` is unsafe

Consider this example:

```
ResourceHolder holder = new ResourceHolder();
holder.set(new Laptop("L-14"));

Book b = (Book) holder.get();
```

The compiler allows the cast syntax. But the stored object is a `Laptop`, not a `Book`.

So the failure moves to runtime as `ClassCastException`.

Important teaching point:

> `Object` reduces duplication, but it also removes useful type information.

7.5 Attempt 3: use a generic class

This is exactly where generics help.

```
class ResourceHolder<T> {
    private T item;

    void set(T item) { this.item = item; }
    T get() { return item; }
}
```

Now we still have:

- one reusable class,

but we also get:

- compile-time type checking.

7.6 How to read the syntax correctly

Step 1: class declaration. `class ResourceHolder<T>` means:

> define a class that works for some type to be chosen later

Step 2: type parameter. The symbol `T` is the **type parameter**. It is only a placeholder name. It does not mean a concrete class by itself.

Step 3: parameterized use. When we write:

```
ResourceHolder<Book> bookHolder;
```

then:

- `Book` is the **type argument**,
- `ResourceHolder<Book>` is the **parameterized type**.

Step 4: mental model. You can read

```
ResourceHolder<Book>
```

as:

> a ResourceHolder whose item type is Book

7.7 Safe usage

```
ResourceHolder<Book> bookHolder = new ResourceHolder<>();
bookHolder.set(new Book("Clean Code"));
Book b = bookHolder.get();
```

Notice what changed compared to the `Object` version:

- no cast is required when calling `get()`,
- the compiler knows the result should be a `Book`,
- a wrong assignment can be caught earlier.

7.8 Why generic classes improve teaching clarity

Generic classes do not only improve safety. They also improve readability.

Compare these two declarations:

```
ResourceHolder rawHolder;
ResourceHolder<Book> bookHolder;
```

The second line communicates design intent immediately. That matters in real projects, especially when classes are used by other developers.

7.9 Common naming conventions

Java commonly uses short uppercase letters for type parameters:

- `T` = Type
- `E` = Element
- `K` = Key
- `V` = Value

These names are conventions, not magic keywords.

7.10 Primitive types do not work directly

This is a frequent beginner confusion.

```
ResourceHolder<int> marks; // invalid
ResourceHolder<Integer> marks; // correct
```

Generics require reference types. Primitive values must be wrapped using classes such as:

- `Integer`
- `Double`
- `Character`
- `Boolean`

7.11 Raw types and why they are dangerous

Raw type means using the class name without a type argument:

```
ResourceHolder raw = new ResourceHolder(new LabKey("LAB-2"));
```

Why Java still allows raw types:

- backward compatibility with old pre-generics code

Why you should usually avoid them:

- they disable the main safety advantage of generics,
- they cause warnings,
- they make later casts risky.

7.12 A useful intuition about compile-time vs runtime

Generics in Java are mainly a **compile-time safety feature**. For this lecture, the key takeaway is simple:

- without generics, many mistakes survive until runtime,
- with generics, many of those mistakes are caught earlier.

That is the pedagogically important point for now. More advanced runtime details such as type erasure will be revisited later.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

8 Running Project Connection

Use Generics Fundamentals (Generic Classes) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Treating generic type parameters as if they were inheritance declarations.
- Falling back to raw types and losing type safety.
- Expecting primitive types to work directly as generic arguments.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

10 Worked Example: campus resources

Domain classes

```java
class Book {
    String title;
    Book(String title) { this.title = title; }
}

class LabKey {
    String roomCode;
    LabKey(String roomCode) { this.roomCode = roomCode; }
}
```

Generic class

```java
class ResourceHolder<T> {
    private final T item;

    ResourceHolder(T item) { this.item = item; }
    T get() { return item; }
}
```

Safe usage

```java
public class DemoSafe {
    public static void main(String[] args) {
        ResourceHolder<Book> b =
            new ResourceHolder<>(new Book("Clean Code"));
        ResourceHolder<LabKey> k =
            new ResourceHolder<>(new LabKey("LAB-2"));

        Book book = b.get();
        LabKey key = k.get();
        System.out.println(book.title + " / " + key.roomCode);
    }
}
```

Unsafe raw-type usage

```java
public class DemoRaw {
    public static void main(String[] args) {
        ResourceHolder raw =
            new ResourceHolder(new LabKey("LAB-2"));

        Book wrong = (Book) raw.get();
        System.out.println(wrong.title);
    }
}
```

What this example teaches.

- The generic version keeps one reusable class.

- Different type arguments create different typed usages.
- The raw version removes the generic benefit and reintroduces risk.

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Why this topic exists* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Why is a single `Object`-based holder better than many repeated holder classes, but still not the best final solution?

   Answer: It removes duplication, but it loses type information and forces unsafe casts.

2. What is the difference between a type parameter and a type argument?

   Answer: The type parameter is the placeholder in the class declaration, such as `T`; the type argument is the real type supplied later, such as `Book`.

3. Why do we write `ResourceHolder<Integer>` instead of `ResourceHolder<int>`?

   Answer: Generic type arguments must be reference types, so we use the wrapper class `Integer` instead of the primitive `int`.

13 Self-Study Practice (no solutions)

1. Create a generic class `Pair<K,V>` that stores two related values.
2. Write a generic `TicketHolder<T>` and instantiate it with two different user-defined classes.
3. Find one old example that uses raw collections and rewrite it with parameterized types.
4. Explain in your own words why "one class with `Object`" is not the same as a generic class.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. What design problem leads programmers toward generic classes?

Sample answer: The same class logic is often needed for many data types. Writing separate classes for each type causes duplication, while using `Object` removes safety. Generic classes solve both issues together.

Q2. What does `ResourceHolder<Book>` mean?

Sample answer: It means a typed version of `ResourceHolder` whose stored item is expected to be a `Book`. Here, `Book` is the type argument.

Q3. Why should modern Java code avoid raw types where possible?

Sample answer: Raw types weaken compile-time checks, create warnings, and make unsafe casts possible again. They remove the main advantage that generics were introduced to provide.